

KLEPSYDRA AI GETTING STARTED GUIDE V25.5.5

23.07.2026

Klepsydra AI product Suite	4
Overview	4
Klepsydra AI	4
Klepsydra GPU Streaming / AI	5
Klepsydra SDO (Streaming Distribution Optimiser)	5
Klepsydra Deployment approach.	6
Installation Instructions	7
Distribution content	7
Installation format	7
Klepsydra AI versions	7
Summary table	7
Tarball file name	7
Docker file name	8
Prerequisites	8
For Klepsydra AI Minimal Legacy:	8
For Klepsydra AI Minimal with Full backend:	8
For Klepsydra AI full:	8
Software binaries installation	8
Tarball installation	8
Docker installation	8
Examples preparation	9
Overview	9
Example compilation and execution (Version Klepsydra AI minimal)	9
Tarball format	9
Docker format	9
Run the example	9
Example compilation and execution (Version Klepsydra AI full)	10
Tarball format	10
Docker format	10
Generating the license	10
Environment Variables	10
License file	11
OpenCV Binding	12
OpenCV Examples	13
Simple example	13
Depth-estimation example	13
Python Binding	14
Compilation and execution of the example	14

Requirements	14
Build	14
Run	14
Configuration parameters	15
Python Binding Explained	15
Setting up the benchmarking tool	16
Configuration:	16
Working with the examples	17
Running the example	17
Configuration Settings	18
Test mode	18
The streaming configuration file	18
Gathering statistics.	20
Part II: The power of Klepsydra AI	22
Basic optimisation	22
Increase data publishing rate	22
Advanced optimisation	23
Appendix A: Configuration	24
Main configuration file	24
Streaming Configuration	24
Appendix B: Third party installation	26
Installation of Protobufs	26
Overview	27
Contact	27

Klepsydra AI product Suite

Overview

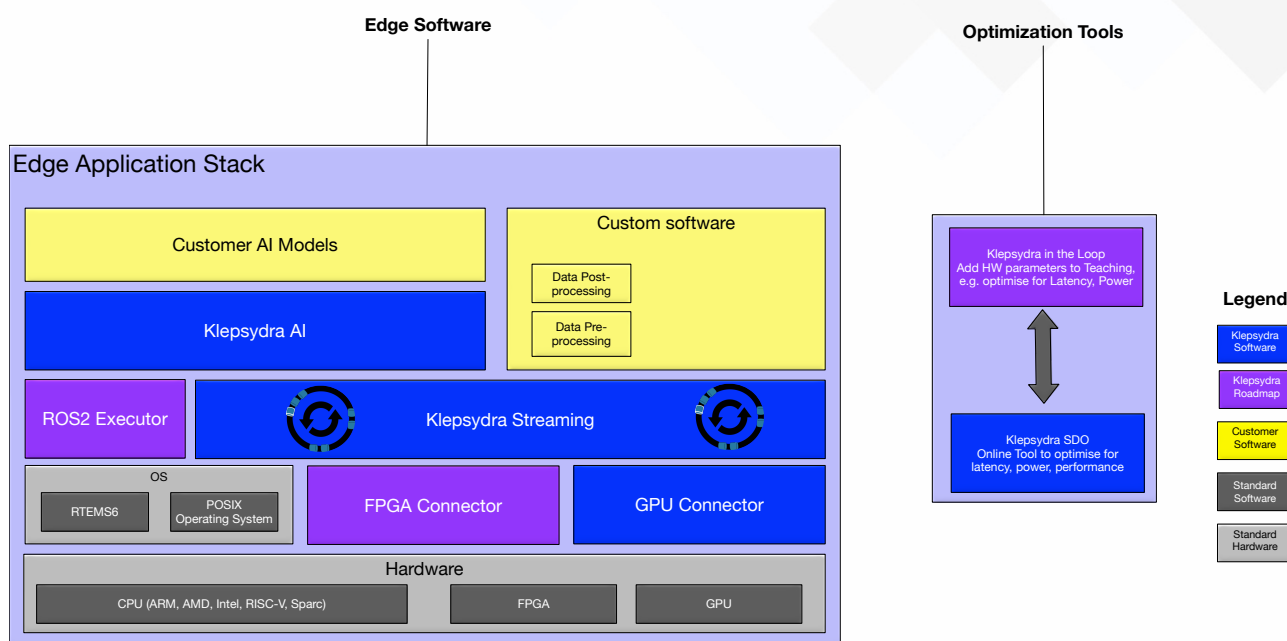


Figure 1: Klepsydra Software Suite

The Klepsydra software suite consists of four main commercially available components:

1. Klepsydra AI
2. Klepsydra Streaming
3. Klepsydra GPU Streaming
4. Klepsydra GPU AI
5. Klepsydra SDO

Additionally, three more components are currently under development:

1. Klepsydra ROS2 Executor
2. FPGA Connector
3. Klepsydra-in-the-Loop (KITL)

These products are dedicated to enabling high-performance application deployment at the edge. This guide specifically focuses on the AI product. The SDO user guide is a related document that is highly relevant to this guide, and users should keep it for reference while using this guide..

Klepsydra AI

Klepsydra AI is a high performance deep neural network engine for edge computers. Customers can deploy existing or new trained models on the edge using Klepsydra AI, in the same manner as with standard edge AI solutions but significant performance boost: 4x more data rate.

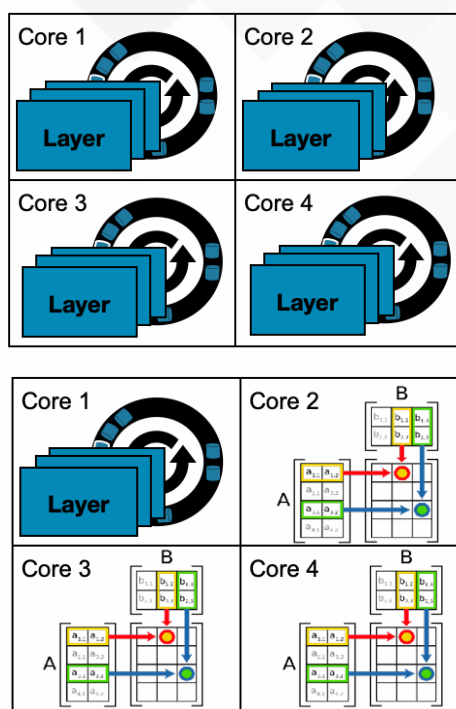


Figure 2. Klepsydra AI combines pipelining and parallelisation using a two dimensional threading model.

Klepsydra GPU Streaming / AI

Klepsydra has developed a new software product for GPU programming called Klepsydra GPU Streaming and GPU AI. Based on Klepsydra Streaming, the occupancy problem is solved very efficiently. Not only that but also is very easy to use. With a simple C++/CUDA, Klepsydra GPU Streaming is compatible with most NVIDIA boards.

Klepsydra GPU streaming is currently patent pending, so we cannot reveal details of its inner workings. However, we are able to share that the benchmarks of the solution applied to large matrix multiplications are outstanding and demonstrate that this new product is capable of increasing the performance of GPU algorithms by applying an intelligent approach to increase occupancy.

Klepsydra GPU AI is an extension library for Klepsydra AI designed to accelerate the execution of Artificial Intelligence (AI) algorithms.

Klepsydra SDO (Streaming Distribution Optimiser)

This software is cloud-based and requires a tarball containing the logs from multiple dry runs on the target computer. Once the logs are uploaded, the software runs an optimizer to determine the optimal solution for latency, power consumption, or throughput. The software then generates a configuration file, which can be copied over to the target computer to implement the optimal solution.

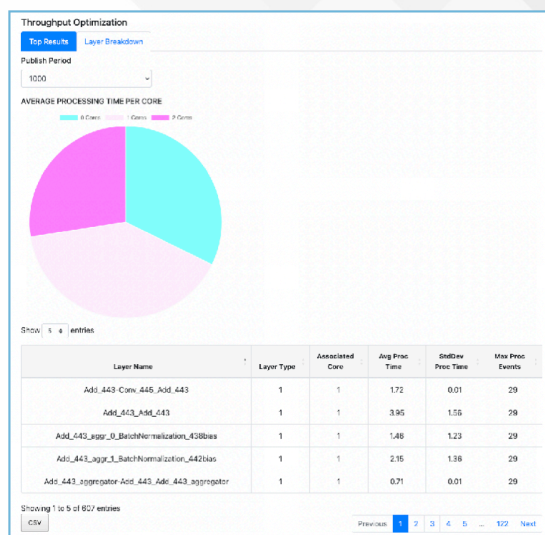


Figure 3. User Interface of Klepsydra SDO.

Klepsydra Deployment approach.

The deployment of AI models with Klepsydra AI can be achieved in several ways. One of the most common methods for high-end edge processors is via Docker containers. By using Docker Compose or similar techniques, customers can deploy their code on top of Klepsydra AI Docker containers.

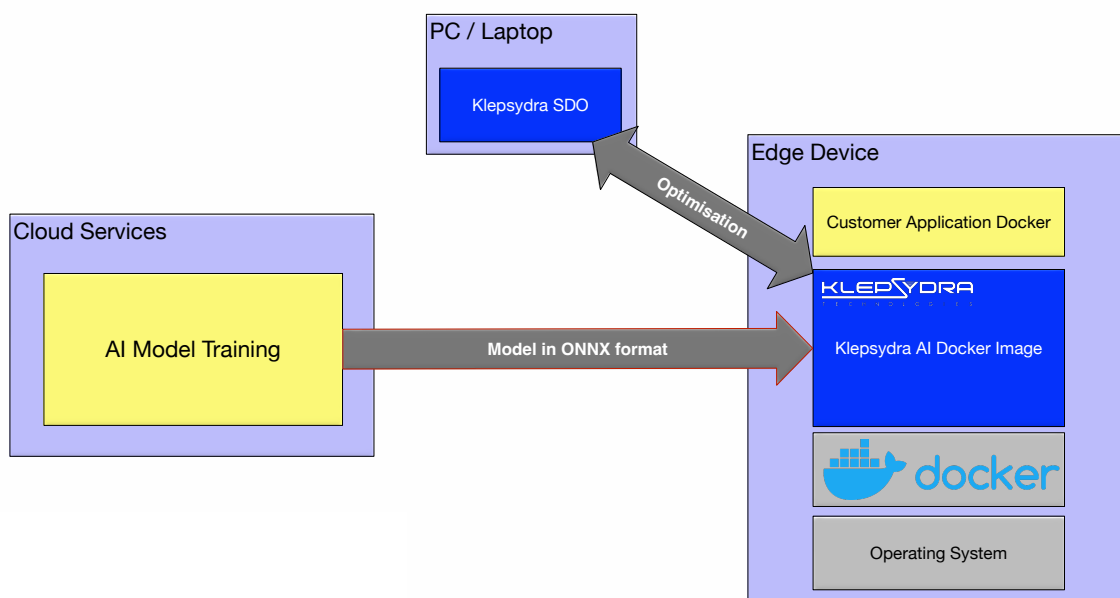


Figure 4: Docker-based deployment approach

A key feature of deploying with Klepsydra AI is the ability to transfer trained models using the standard ONNX format. This ensures seamless integration and deployment.

Finally, once the model has been tested at the edge, the SDO can be utilised to optimise execution performance using Klepsydra AI.

Installation Instructions

Distribution content

Installation format

Klepsydra AI comes in three installation formats:

- Docker image. An image is provided with all the necessary software and corresponding dependencies.
- Tarballs. Several tarballs to be uncompressed in the desired target board folder.

Klepsydra AI versions

On the other hand, Klepsydra AI comes in three different variants:

A. Klepsydra Full Support:

- With support to load ONNX files directly.
- Klepsydra full mathematical backend support. Standard for modern processors and modern GCC compilers (7.5.0 and above).

B. Klepsydra Minimal with Full Mathematical backend:

- With support to no third party library. The minimal version is aimed at target edge devices with limited operating system support.
- Klepsydra full mathematical backend support. Standard for modern processors and modern GCC compilers (7.5.0 and above).

C. Klepsydra Minimal Legacy:

- With support to no third party library. The minimal version is aimed at target edge devices with limited operating system support.
- Klepsydra quantised mathematical backend support. Aimed at older processors and older GCC compilers (before 7.5.0).

Summary table

Different variants and version of the software will come with different names. For Docker format, all these files come already pre-installed.

Format	Docker	Tarball	GCC Support	ONNX Support
Full	Yes	Yes	7.5.0+	Yes
Minimal Full math backend	No	Yes	7.5.0+	No
Minimal Legacy	No	Yes	5.x	No

Tarball file name

The tarball name follows this convention:

`kpe-ai[format]_sdk_[version]_[architecture].tar.gz`

Where:

- Format can be empty or '-minimal' for minimal build release
- Version is in the format x.y.z (e.g., 25.4.0)
- Architecture is the CPU architecture (e.g., rpi4, rpi3, rpi5, aarch64 or amd64)

Docker file name

The tarball name follows this convention:

```
kpe-ai_docker_[version]_[architecture].tar.gz
```

Where:

- Version is in the format x.y.z (e.g., 25.4.0)
- Architecture is the CPU architecture (e.g., rpi4, rpi3, rpi5, aarch64 or amd64)

Prerequisites

For Klepsydra AI Minimal Legacy:

- cmake 3.16 and above
- build-essential
- GCC 5.x and above.

For Klepsydra AI Minimal with Full backend:

- cmake 3.16 and above
- build-essential
- GCC 7.5.0 and above.

For Klepsydra AI full:

- cmake 3.16 and above
- build-essential
- GCC 7.5.0 and above.
- Protobufs 3.19.0

Notes:

The installation of the third-party software components Protobufs is the appendix B.

The requirement to install protobufs will disappear in a future release.

Software binaries installation

Tarball installation

Unzip the software packages to a folder of your choice, e.g.: /home/\$USER/klepsydra

```
cd /home/$USER/klepsydra
```

```
tar xvf /home/$USER/kpe-ai-minimal_sdk_25.4.0_rpi4.tar.gz
```

This will uncompress the software in the folder /home/\$USER/klepsydra/usr/local¹

Docker installation

Load into docker the image provided by Klepsydra²:

```
docker load < kpe-ai_docker_25.4.0_rpi4.tar.gz
```

¹ There is currently an issue in the installation setup, that requires that all the files are copied to the /usr/local folder. Therefore, once uncompressed do the following command: `sudo cp -r usr/local/* /usr/local`

² The image name might vary from one edge computer to another. Please check the correct name from the email received from Klepsydra with the download instructions.

Examples preparation

Overview

The examples included in this release cover the available API of Klepsydra AI. The API of Klepsydra AI depends on the version, minimal or full:

Example	Minimal	Full	Required Third parties	Features used
ai-min-example	Yes	No		Core Features
ai-min-raw-images-example	Yes	No		Core features, raw images in file
ai-min-onnx-example	No	Yes		Core Features
ai-onnx-example	No	Yes	OpenCV	ONNX Importer
depth-estimation-example	No	Yes	OpenCV	ONN Importer, OpenCV Binding
ai-python-example	No	Yes	Python, OpenCV	ONNX Importer, Python Binding

Notes:

The depth estimation example, also needs two Debian packages for the model and the data respectively. These can be provided upon request to the support@klepsydra.com.

Example compilation and execution (Version Klepsydra AI minimal)

Tarball format

Compile one the examples in the examples folder (e.g. the minimal example):

```
cd /home/$USER/klepsydra/
cd kpe-ai-examples/ai-min-onnx-example/
mkdir build
cd build/
cmake -DCMAKE_INSTALL_PREFIX=/home/$USER/klepsydra/usr/local/lib/cmake/ ..
make
```

[Optional] you may want to verify that the library was correctly compiled, and all the required deps are found:

```
ldd ./kpsr_ai_min_example
```

Docker format

Copy the examples from the /usr/local/shared/klepsydra/examples to your home:

```
cp -r /usr/local/src/kpe-ai-examples /home/$USER/klepsydra
```

Compile one the examples in the examples folder (e.g. the minimal example):

```
cd /home/$USER/klepsydra/kpe-ai-examples/ai-min-example/
mkdir build
cd build/
cmake ..
make
```

Run the example

```
./ai-min-example -f ../conf/mem_dnn_conf.json -m ../models/alexnet.kpsr
```

Example compilation and execution (Version Klepsydra AI full)

Tarball format

Compile one the examples in the examples folder (e.g. the onnx example):

```
cd /home/$USER/klepsydra/
cd kpe-ai-examples/ai-min-onnx-example/
mkdir build
cd build/
cmake -DCMAKE_INSTALL_PREFIX=/home/$USER/klepsydra/usr/local/lib/cmake/ ..
make
```

[Optional] you may want to verify that the library was correctly compiled, and all the required deps are found:

```
ldd ./ai-min-onnx-example
```

Run the example

```
./ai-min-onnx-example -f ../conf/mem_dnn_conf.json -m ../models/alexnet.onnx
-d ../imgs/
```

Docker format

Copy the examples from the /usr/local/shared/klepsydra/examples to your home:

```
cp -r /usr/local/src/kpe-ai-examples /home/$USER/klepsydra/
cd kpe-ai-examples/ai-min-onnx-example/
mkdir build
cd build/
cmake ..
make
```

Run the example

```
./ai-min-onnx-example -f ../conf/mem_dnn_conf.json -m ../models/alexnet.onnx
-d ../imgs/
```

Generating the license

Environment Variables

Three environment variables are required for running Klepsydra libraries:

Env Variable	Example Value	Description
LD_LIBRARY_PATH	/home/\$USER/klepsydra/ usr/local: \$LD_LIBRARY_PATH	Location of the shared objects of Klepsydra. Only needed for tarball installation
IDENTIFICATION_STRATEGY	STRATEGY_HOST_NAME	Required by Klepsydra License Manager
KPSR_LICENSE_FILE	/home/\$USER/klepsydra/ klepsydra.lic	Location of the provided license file. This file will provided after the installation has been completed. For the installation is not required. In the case of docker execution, this value is expected to be /opt/klepsydra/klepsydra.lic

Example:

```
export LD_LIBRARY_PATH=/home/$USER/klepsydra/usr/local:$LD_LIBRARY_PATH
export IDENTIFICATION_STRATEGY=STRATEGY_HOST_NAME
export KPSR_LICENSE_FILE=/home/$USER/klepsydra/klepsydra.lic
```

License file

After the first execution of the example (either minimal or full) you will get an output like this:

```
[Fri Dec 22 08:54:49 2023] [08:54:49 169857] [klepsydra] [error] [38860] License
Error: this product was not licensed
[Fri Dec 22 08:54:49 2023] [08:54:49 169876] [klepsydra] [error] [38860] Please
provide the path to the license file in
[Fri Dec 22 08:54:49 2023] [08:54:49 169878] [klepsydra] [error] [38860] the
KPSR_LICENSE_FILE environment variable.
[Fri Dec 22 08:54:49 2023] [08:54:49 169881] [klepsydra] [error] [38860] If the
environment variable is not provided, klepsydra.lic will be used.
[Fri Dec 22 08:54:49 2023] [08:54:49 170087] [klepsydra] [error] [38860]
-----
[Fri Dec 22 08:54:49 2023] [08:54:49 170095] [klepsydra] [error] [38860] PC
Signature: XXXX-XXXX-XXXX
[Fri Dec 22 08:54:49 2023] [08:54:49 170098] [klepsydra] [error] [38860]
-----
```

This should print an license error with a PC Signature that you have to provide to Klepsydra via email (to sales@klepsydra.com)

A file named klepsydra.lic will be provided by email and needs to be copied to the target processor to some location. The path of this file will be used to set the corresponding environment variable as explained above.

OpenCV Binding

Klepsydra AI provides source code for an example binding to OpenCV. This binding allows developers to pass `cv::Mat` objects directly to the predict method and receive `cv::Mat` objects in the callback function. While a reference API is provided, developers are free to adapt both the implementation in the source file and the API itself to suit their specific needs.

```
#ifndef OCV_DNN_H
#define OCV_DNN_H

#include <memory>

#include <opencv2/opencv.hpp>

#include <klepsydra/ai/dnn.h>

namespace kpsr {
namespace ai {
namespace OpenCVDeepNeuralNetwork {

/**
 * @brief Sets the output callback for when an event is received.
 *
 * @note The cv::Mat returned by the dnn is a single channel vector.
 * In case your network returns multiple predictions, the result is flattened and the user should
ensure that it is
 * reshaped to the right dimensions by your callback function.
 *
 * @param dnn DNN to set the callback.
 * @param callback Callback function to be invoked for the prediction result.
 */
KPSR_API void setCallback(DeepNeuralNetwork &dnn,
                          std::function<void(const unsigned long &, const cv::Mat &)> callback);

/**
 * @brief Loads and runs a DNN with a specific input image.
 *
 * @note In case of a 3 channel image, kpsr-ai expects HWC data format similar to OpenCV and data is
loaded directly.
 * Note that channel order is not enforced by kpsr-ai (RGB vs BGR) and the user should ensure that
it matches the
 * order expected by the network (OpenCV images are in BGR channel order by default).
 *
 * @param dnn DNN on which to run the prediction.
 * @param inputBlob `cv::Mat` containing the input image data.
 *
 * @return Unique ID of the input image.
 */
KPSR_API unsigned long predict(DeepNeuralNetwork &dnn, const cv::Mat &inputBlob);

/**
 * @brief Loads and runs a DNN with a specific input image associated with an id.
 *
 * @note In case of a 3 channel image, kpsr-ai expects HWC data format similar to OpenCV and data is
loaded directly.
 * Note that channel order is not enforced by kpsr-ai (RGB vs BGR) and the user should ensure that
it matches the
 * order expected by the network (OpenCV images are in BGR channel order by default).
 *
 * @param dnn DNN on which to run the prediction.
 * @param inputBlob `cv::Mat` containing the input image data.
 * @param id Unique ID of the input image.
 *
 * @return Unique ID of the input image.
 */
KPSR_API void predict(DeepNeuralNetwork &dnn, const cv::Mat &inputBlob, unsigned long id);

} // namespace OpenCVDeepNeuralNetwork
} // namespace ai
} // namespace kpsr

#endif // OCV_DNN_H
```

OpenCV Examples

Under the kpe-ai-examples folder, there is a folder “ai_opcv” that contains the source code of the binding:

- kpe-ai-examples/ai_opcv/include/ocv_dnn.h and
- kpe-ai-examples/ai_opcv/src/ocv_dnn.cpp

Simple example

Compile one the examples in the examples folder (e.g. the onnx example):

```
cd /home/$USER/klepsydra/kpe-ai-examples/ai-onnx-example/
mkdir build
cd build/
cmake ..
make
```

Run the example

```
./ai-onnx-example -f ../conf/mem_dnn_conf.json -m ../models/alexnet.onnx -d ../imgs/
```

Depth-estimation example

As part of the release packages, a more advanced example is provided that combines Klepsydra AI with OpenCV pre and post processing. This example can serve as a base for further development of AI applications using Klepsydra AI and OpenCV.

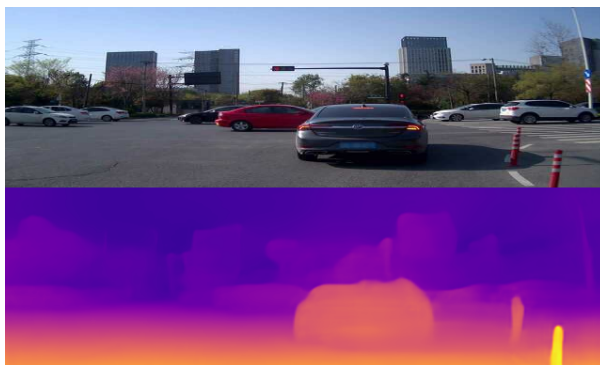


Figure 5: Output image of the depth estimation model

In order to run this model, two additional packages need to be installed (only available via Debian packages)

```
kpe-depth-estimation-demo-model_0.0.1_all.deb
kpe-depth-estimation-demo-testset_0.0.1_all.deb
```

The compilation and execution are quite similar to the previous examples:

```
cd /home/$USER/klepsydra/kpe-ai-examples/depth-estimation-example
mkdir build
cd build
cmake ..
make
```

For execution:

```
mkdir output (folder where the output images will be written)
./depth-estimation-example -f ../conf/monodepth_conf_kpsr.json -m /usr/local/
share/kpe-depth-estimation-demo/models/monodepthencoder_decoder_v2_named.onnx
-d /usr/local/share/kpe-depth-estimation-demo/images -o output -t 10000
```

Python Binding

An example includes the source code of the Klepsydra AI binding to Python. This binding gives developers the option to use Python to create AI applications using Klepsydra AI inference engine. While a reference API is provided, developers are free to adapt both the implementation in the source file and the API itself to suit their specific needs.

Compilation and execution of the example

This example runs the MobileNet DNN on a set of images of birds. The purpose is to show how Klepsydra AI outperforms in terms of latency and throughput.

This model was pre-trained for classifying objects and animals and can appropriately label images of birds.

The code is the Python port of the `ai-onnx-example` written in C++.

For your convenience, the example includes the Klepsydra AI Python binding source code, to be built in the Python environment of your choice.

Requirements

Install the Python environment of your choice, e.g.

```
sudo apt install python3 python3-opencv
```

Build

Copy the source files to a writable folder, where you can edit, and build the Klepsydra Python binding with CMake:

```
cp -a /usr/local/src/kpe-ai-examples/ai-python-example/ .
cd ai-python-example/kpe_ai/
cmake .
make
cd ../
```

Once built, the Klepsydra AI Python module consists of two files:

- kpe_ai/__init__.py and
- kpe_ai/kpe_ai.cpython-<python_version>-x86_64-linux-gnu.so

Besides the kpe_ai/ module folder, the example contains the Python code and the app configuration. The neural network model (in ONNX format) and sample images are borrowed from the ai-onnx-example.

Run

Set the license environment variables:

```
export IDENTIFICATION_STRATEGY=0
export KPSR_LICENSE_FILE=/usr/local/share/licenses/klepsydra.lic
```

The first lines of the application code contains the recommended options:

```
ONNX_MODEL_FILE = '/usr/local/src/kpe-ai-examples/ai-onnx-example/models/mobilenetv1.onnx'
KPSR_CONF_FILE = './mem_dnn_conf.json'
IMAGES_PATH = '/usr/local/src/kpe-ai-examples/ai-onnx-example/images/'
INTERVAL_IN_MS = 1000
TIMEOUT_IN_S = 30
```

INTERVAL_IN_MS defines the feed rate for images provided as input.

Run the example:

```
python3 ai-python-example.py
```

The console outputs the inference steps:

Applying AI to image 000.jpg

Image 0 is classified as 452

Processed 1 of 10

Applying AI to image 001.jpg

...

Configuration parameters

File `mem\_dnn\_conf.json` collects the parameters that define the application behaviour as explained in Appendix A.

Python Binding Explained

Going line by line on the python example:

```
1. def prediction_callback(image_id, prediction_output):
2.     global processed_image_counter, images_number
3.     print(f'Image {image_id} is classified as {np.argmax(prediction_output)}')
4.     processed_image_counter += 1
5.     print(f'Processed {processed_image_counter} of {images_number}')
6.
7.
8. processed_image_counter = 0
9. images_list = sorted(os.listdir(IMAGES_PATH))
10. images_number = len(images_list)
11. print(f'Read a list of {images_number} images in folder: {IMAGES_PATH}')
12.
13. with kpe_ai.Dnn(ONNX_MODEL_FILE, KPSR_CONF_FILE) as dnn:
14.     if dnn.get_pointer() is None:
15.         print('Failed to load the DNN model')
16.         exit(-1)
17.
18.     dnn.set_callback(prediction_callback)
19.
20.     for image_name in images_list:
21.         img = cv2.imread(os.path.join(IMAGES_PATH, image_name))
22.         print(f'Applying AI to image {image_name}')
23.         input_scaled_vector = np.array(img).flatten().astype(np.float32) / 255.0
24.         dnn.predict(input_scaled_vector)
25.         time.sleep(INTERVAL_IN_MS / 1000)
26.
27.     for timeout in range(TIMEOUT_IN_S):
28.         if processed_image_counter < images_number:
29.             time.sleep(1)
30.         else:
31.             break
32.     else:
33.         print(
34.             f'Processed only {processed_image_counter} of {images_number}. '
35.             f'Increase TIMEOUT_IN_S to wait for all images to be processed and also consider
             increasing the INTERVAL_IN_MS')
```

- Lines 1-5: Definition of the callback function. Please notice that the API is the same as the C++ API
- Lines 13: Load the AI algorithm and create the DNN Object. As opposed to the C++ version, [this API supports both ONNX and KPSR Files](#).
- Line 18: Callback registration.
- Line 24: Predict call. This function returns an id that is used to correlate the result from the callback in the same manner that is done in C++
- Line 21: is equivalent to the OpenCV binding in C++.

Setting up the benchmarking tool

As part of the release, a benchmarking tool is included that is used by the SDO to carry out the optimisation of the execution of AI models. This tool includes a shell script that has to be configured similarly to environment variables needed the execution of the examples.

Note: modification of these properties is only needed for tarball installation. Otherwise, use the default properties.

The same variables explained earlier:

Env Variable	Example Value	Description
LD_LIBRARY_PATH	/home/\$USER/klepsydra/usr/local:\$LD_LIBRARY_PATH	Location of the shared objects of Klepsydra. Only need for tarball installation.
IDENTIFICATION_STRATEGY	STRATEGY_HOST_NAME	Required by Klepsydra License Manager. Leave default value for all installation formats.
KPSR_LICENSE_FILE	/home/\$USER/klepsydra/klepsydra.lic	Location of the provided license file. This file will provided after the installation has been completed. For the installation is not required. In the case of docker execution, this value is expected to be /opt/klepsydra/klepsydra.lic

Configuration:

```
#!/bin/bash
```

```
export LD_LIBRARY_PATH=/home/$USER/klepsydra/usr/local:$LD_LIBRARY_PATH
export IDENTIFICATION_STRATEGY=STRATEGY_HOST_NAME
export KPSR_LICENSE_FILE=/home/$USER/klepsydra/klepsydra.lic
```

```
CONF_FOLDER=$1
```

```
BENCHMARK_BINARY=$2
```

Working with the examples

Running the example

Let's start by running the min_onnx example:

```
kpsruser@bb1a0bb9f409:~/examples/computer_vision/min_onnx/build$ ./kpsr_ai_min_onnx_example -f ../conf/mem_dnn_conf.json -m ../../models/alexnet.onnx
```

This should yield an output as shown below:

```
inputData.configurationFile: ../conf/mem_dnn_conf.json
inputData.modelFile: ../../models/alexnet.onnx
[2024-07-09 11:48:22.935] [info] Converting network from onnx format to Klepsydra format
[2024-07-09 11:48:22.965] [info] Network expects input of shape [batch = 1, 224 x 224 x 3] (with channels as the innermost (last) dimension)
[2024-07-09 11:48:22.980] [warning] Max pool layer has kernel_size = 1. Max pool layer will be skipped
[2024-07-09 11:48:23.067] [warning] Max pool layer has kernel_size = 1. Max pool layer will be skipped
[2024-07-09 11:48:23.073] [info] Folding clip layers
[2024-07-09 11:48:23.073] [info] Layers to delete: StatefulPartitionedCall/functional_1/re_lu/Relu
[2024-07-09 11:48:23.073] [info] The layer StatefulPartitionedCall/functional_1/re_lu/Relu has been deleted
[2024-07-09 11:48:23.074] [info] Layers to delete: StatefulPartitionedCall/functional_1/re_lu_1/Relu
[2024-07-09 11:48:23.074] [info] The layer StatefulPartitionedCall/functional_1/re_lu_1/Relu has been deleted
[2024-07-09 11:48:23.074] [info] Layers to delete: StatefulPartitionedCall/functional_1/re_lu_2/Relu
[2024-07-09 11:48:23.074] [info] The layer StatefulPartitionedCall/functional_1/re_lu_2/Relu has been deleted
[2024-07-09 11:48:23.074] [info] Layers to delete: StatefulPartitionedCall/functional_1/re_lu_3/Relu
[2024-07-09 11:48:23.074] [info] The layer StatefulPartitionedCall/functional_1/re_lu_3/Relu has been deleted
[2024-07-09 11:48:23.074] [info] Layers to delete: StatefulPartitionedCall/functional_1/re_lu_4/Relu
[2024-07-09 11:48:23.074] [info] The layer StatefulPartitionedCall/functional_1/re_lu_4/Relu has been deleted

Applying AI to dummy image 0 of 100
Image : 0 assigned id : 0
  Use this id in callback to match input to result.
Applying AI to dummy image 1 of 100
Image : 1 assigned id : 1
  Use this id in callback to match input to result.
Got result for input with id: 0
Processed 0 of 100
Applying AI to dummy image 2 of 100
Image : 2 assigned id : 2
  Use this id in callback to match input to result.
Got result for input with id: 1
Processed 1 of 100
Applying AI to dummy image 3 of 100
Image : 3 assigned id : 3
  Use this id in callback to match input to result.
Applying AI to dummy image 4 of 100
Image : 4 assigned id : 4
  Use this id in callback to match input to result.
Got result for input with id: 2
Processed 2 of 100
...
```

Now, let's have a look at the current configuration file (if you are in the newly created lib folder, the file is found here: ../conf/mem_dnn_conf.json):

```
{
  "StringProperties": {
    "container_name": "",
    "log_filename": "",
    "stat_filename": "",
    "processor_intensive_layers": "conv",
    "streaming_configuration": ""
  },
  "IntProperties": {
    "log_level": 2,
```

```

    "stat_admin_port": 0,
    "stat_system_port": 0,
    "stat_log_interval_ms": 1000,
    "pool_size": 0,
    "number_of_cores": 1,
    "critical_thread_pool_size": 4,
    "non_critical_thread_pool_size": 2,
    "number_of_parallel_threads": 0,
    "first_parallel_core": 0
  },
  "BoolProperties": {
    "use_policy_by_ratio": false,
    "test_mode": false,
    "export_through_tftp": false,
    "export_layer_connections": false,
    "stat_socket_container_enable": false,
    "stat_file_container_enable": false,
    "use_default_streaming_factory": true,
    "export_streaming_configuration": false
  }
}

```

Configuration Settings

The file explained here can be found in each example under the 'conf' folder. Further information of the configuration files can be found in Appendix A.

Test mode

The test mode is made for testing the execution in a single-threaded manner³. It is available for testing the compatibility of the provided AI model file with Klepsydra AI, e.g., are all layers, supported, are the results correct, etc. It should not be used for actual execution of the inference, but for verifying that the model works correctly:

```

{
  "StringProperties": {
    "container_name": "",
    "log_filename": "",
    "stat_filename": "",
    "processor_intensive_layers": "conv",
    "streaming_configuration": ""
  },
  "IntProperties": {
    "log_level": 2,
    "stat_admin_port": 0,
    "stat_system_port": 0,
    "stat_log_interval_ms": 1000,
    "pool_size": 0,
    "number_of_cores": 1,
    "critical_thread_pool_size": 4,
    "non_critical_thread_pool_size": 2,
    "number_of_parallel_threads": 0,
    "first_parallel_core": 0
  },
  "BoolProperties": {
    "use_policy_by_ratio": false,
    "test_mode": true,
    "export_through_tftp": false,
    "export_layer_connections": false,
    "stat_socket_container_enable": false,
    "stat_file_container_enable": false,
    "use_default_streaming_factory": true,
    "export_streaming_configuration": false
  }
}

```

The streaming configuration file

One of the pivotal features of Klepsydra AI is its capability to distribute workload evenly across orchestrating cores that execute kernels.

³ For very large models, the test mode might produce a stack overflow error when embedded resources are limited. A new version of the test mode is under design for a near future release.

Generate a default configuration file by setting `export_streaming_configuration` to true and specify a file path for `streaming_configuration` in order to run the example:

```
{
  "StringProperties": {
    "container_name": "",
    "log_filename": "",
    "stat_filename": "",
    "processor_intensive_layers": "conv",
    "streaming_configuration": "./default_streaming_configuration.json"
  },
  "IntProperties": {
    "log_level": 2,
    "stat_admin_port": 0,
    "stat_system_port": 0,
    "stat_log_interval_ms": 1000,
    "pool_size": 0,
    "number_of_cores": 1,
    "critical_thread_pool_size": 4,
    "non_critical_thread_pool_size": 2,
    "number_of_parallel_threads": 0,
    "first_parallel_core": 0
  },
  "BoolProperties": {
    "use_policy_by_ratio": false,
    "test_mode": true,
    "export_through_tftp": false,
    "export_layer_connections": false,
    "stat_socket_container_enable": false,
    "stat_file_container_enable": false,
    "use_default_streaming_factory": true,
    "export_streaming_configuration": true
  }
}
```

A file will be created with the following content:

```
{
  "version_number": 3,
  "pool_size": 0,
  "number_of_cores": 1,
  "number_of_event_loops": 1,
  "number_of_parallel_threads": 0,
  "first_parallel_core": 0,
  "critical_thread_pool_size": 4,
  "non_critical_thread_pool_size": 2,
  "parallelised_layers": [],
  "event_loop_core_map": {
    "0": [
      0
    ]
  },
  "layer_event_loop_map": {
    "StatefulPartitionedCall/functional_1/conv2d/BiasAdd": {
      "coreId": 0,
      "factoryType": 1
    },
    "StatefulPartitionedCall/functional_1/conv2d_1/BiasAdd": {
      "coreId": 0,
      "factoryType": 1
    },
    "StatefulPartitionedCall/functional_1/conv2d_2/BiasAdd": {
      "coreId": 0,
      "factoryType": 1
    },
    "StatefulPartitionedCall/functional_1/conv2d_3/BiasAdd": {
      "coreId": 0,
      "factoryType": 1
    },
    "StatefulPartitionedCall/functional_1/conv2d_4/BiasAdd": {
      "coreId": 0,
      "factoryType": 1
    },
    "StatefulPartitionedCall/functional_1/max_pooling2d/MaxPool": {
      "coreId": 0,
      "factoryType": 1
    },
    "first": {

```

```

        "coreId": 0,
        "factoryType": 1
    }
}
}

```

The content of this file is explained in Appendix A, so it is not going to be repeated here. However, this file is required for advanced optimisation of the model execution on the edge, adequate configuration of this file can achieve substantial performance increase with respect to the default configuration generated by Klepsydra. This file is used by the SDO to find optimal configurations for execution of AI for latency and throughput.

Please note that the other properties were copied from the original configuration file.

Gathering statistics.

System statistics can be collected using Klepsydra's internal statistics collector, which is Klepsydra SDO to calculate the optimal configuration. This statistics can be created by not only Klepsydra AI but by all products of Klepsydra that are deployed at the edge (Streaming, GPU Connector, etc).

To enable this feature, modify the configuration file as follows:

```

{
  "BoolProperties": {
    "export_streaming_configuration": false,
    "test_mode": false,
    "use_default_streaming_factory": true,
    "stat_file_container_enable": true
  },
  "IntProperties": {
    "number_of_parallel_threads": 0,
    "first_parallel_core": 0,
    "critical_thread_pool_size": 3,
    "cuda_device_id": 0,
    "device_stream_callback_pool_size": 2,
    "log_level": 2,
    "non_critical_thread_pool_size": 1,
    "number_of_parallel_streams": 1,
    "number_of_cores": 1,
    "number_of_event_loops": 2,
    "stat_log_interval_ms": 100,
    "pool_size": 0
  },
  "StringProperties": {
    "log_filename": "",
    "streaming_configuration": "./default_streaming_configuration.json",
    "container_name": "helloworld",
    "stat_filename": "./helloworld_stats.csv"
  }
}

```

This will generate a file like this. While these statistics may not be human-readable, they are utilised with Klepsydra wrapper of OpenMCT for real-time performance monitoring and by SDO for performance optimisation:

```

1720525923450,cpuData,79,36,ramdata,1850064896,1772060672,78004224,57032704,managedServices,0,servic
es,0,publishers,7,0_StatefulPartitionedCall/functional_1/conv2d_1/
BiasAdd,156,156,0,0,EVENT_LOOP,0,0,0_StatefulPartitionedCall/functional_1/co
nv2d/BiasAdd,157,157,2,35124,EVENT_LOOP,0,3,0_StatefulPartitionedCall/functional_1/max_pooling2d/
MaxPool,156,156,1,15000,EVENT_LOOP,0,1,0_StatefulPartitionedCall/functional_1/conv2d_2/
BiasAdd,156,156,0,0,EVENT_LOOP,0,0,0_StatefulPartitionedCa
ll/functional_1/conv2d_3/BiasAdd,156,156,0,0,EVENT_LOOP,0,0,0_StatefulPartitionedCall/functional_1/
c
o
n
v
2
d
4
/
BiasAdd,156,156,0,0,EVENT_LOOP,0,0,0_first,157,157,3,32499,EVENT_LOOP,0,0,subscribers,14,StatefulPar
titionedCall/functional_1/max_pool
ing2d/MaxPool_external_StatefulPartitionedCall/functional_1/max_pooling2d/
MaxPool_external,156,156,0,0,UNSAFE_MULTI_EVENT_EMITTER,0,0,StatefulPartitionedCall/functional_1/
conv2d_1/BiasAdd-StatefulPartitionedCall/functional_1/conv2d_2/BiasAdd_
S
t
a
t
e
f
u
l
P
a
r
t
i
t
i
o
n
e
d
C
a
l
l
/
f
u
n
c
t
i
o
n
a
l
_
1
/
c
o
n
v
2
d
_
1
/
BiasAdd,156,156,0,0,UNSAFE_MULTI_EVENT_EMITTER,0,0,first-StatefulPartitionedCall/functional_1/

```

```

BiasAdd_first,156,156,2,104424760,UNSAFE_MULTI_EVENT_EMITTER,30926198,0,StatefulPartitionedCall
/functional_1/conv2d_2/BiasAdd_external_StatefulPartitionedCall/functional_1/conv2d_2/
BiasAdd_external,156,156,0,0,UNSAFE_MULTI_EVENT_EMITTER,0,0,StatefulPartitionedCall/functional_1/
conv2d_3/BiasAdd_external_StatefulPartitionedCall/functional_
1
1
/
c
o
n
v
2
d
_
3
/
BiasAdd_external,156,156,0,0,UNSAFE_MULTI_EVENT_EMITTER,0,0,first_external_first_external,157,157,2,
104456510,UNSAFE_MULTI_EVENT_EMITTER,30911698,0,StatefulPartitionedCall/functional_1/conv2d/BiasAdd-
StatefulPartitionedCall/funct
ional_1/max_pooling2d/MaxPool_StatefulPartitionedCall/functional_1/conv2d/
BiasAdd,156,156,1,1496724,UNSAFE_MULTI_EVENT_EMITTER,48460260,0,callback_StatefulPartitionedCall/
functional_1/conv2d_4/BiasAdd,155,155,0,0,UNSAFE_MULTI_EVENT_EMITTER,0,
0,StatefulPartitionedCall/functional_1/conv2d_4/BiasAdd_external_StatefulPartitionedCall/
f
u
n
c
t
i
o
n
a
l
_
1
/
c
o
n
v
2
d
_
4
/
BiasAdd_external,156,156,0,0,UNSAFE_MULTI_EVENT_EMITTER,0,0,StatefulPartitionedCall/functional_1/
conv2d/BiasAdd_external_StatefulPa
r
t
i
t
i
o
n
e
d
C
a
l
l
/
f
u
n
c
t
i
o
n
a
l
_
1
/
c
o
n
v
2
d
/
BiasAdd_external,156,156,1,1504224,UNSAFE_MULTI_EVENT_EMITTER,48454510,0,StatefulPartitionedCall/
functional_1/conv2d_1/BiasAdd_external_StatefulPartitionedCall/functional_1/conv2d_1/
BiasAdd_external,156,156,0
,0,UNSAFE_MULTI_EVENT_EMITTER,0,0,StatefulPartitionedCall/functional_1/max_pooling2d/MaxPool-
StatefulPartitionedCall/functional_1/conv2d_1/BiasAdd_StatefulPartitionedCall/functional_1/
max_pooling2d/MaxPool,156,156,0,0,UNSAFE_MULTI_EVENT_EMITT
ER,0,0,StatefulPartitionedCall/functional_1/conv2d_3/BiasAdd_StatefulPartitionedCall/functional_1/
c
o
n
v
2
d
_
4
/
B
i
a
s
A
d
d
_
S
t
a
t
e
f
u
l
P
a
r
t
i
t
i
o
n
e
d
C
a
l
l
/
f
u
n
c
t
i
o
n
a
l
_
1
/
c
o
n
v
2
d
_
3
/
BiasAdd,156,156,0,0,UNSAFE_MULTI_EVENT_EMITTER,0,0,StatefulPartitionedCall/functi
o
n
a
l
_
1
/
c
o
n
v
2
d
_
2
/
B
i
a
s
A
d
d
_
S
t
a
t
e
f
u
l
P
a
r
t
i
t
i
o
n
e
d
C
a
l
l
/
f
u
n
c
t
i
o
n
a
l
_
1
/
c
o
n
v
2
d
_
3
/
B
i
a
s
A
d
d
_
S
t
a
t
e
f
u
l
P
a
r
t
i
t
i
o
n
e
d
C
a
l
l
/
f
u
n
c
t
i
o
n
a
l
_
1
/
c
o
n
v
2
d
_
2
/
BiasAdd,156,156,0,0,UNSAFE_MULTI_EVENT_EMITTER,0,0,
1720525923550,cpuData,76,34,ramdata,1850064896,1772318720,77746176,57364480,managedServices,0,servic
es,0,publishers,7,0_StatefulPartitionedCall/functional_1/conv2d_1/
BiasAdd,257,257,0,0,EVENT_LOOP,0,1,0_StatefulPartitionedCall/functional_1/co
nv2d/BiasAdd,258,258,3,49124,EVENT_LOOP,0,3,0_StatefulPartitionedCall/functional_1/max_pooling2d/
MaxPool,257,257,2,83999,EVENT_LOOP,0,2,0_StatefulPartitionedCall/functional_1/conv2d_2/
BiasAdd,257,257,0,0,EVENT_LOOP,0,0,0_StatefulPartitionedCa
ll/functional_1/conv2d_3/BiasAdd,257,257,0,0,EVENT_LOOP,0,0,0_StatefulPartitionedCall/functional_1/
f
u
n
c
t
i
o
n
a
l
_
1
/
c
o
n
v
2
d
_
4
/
BiasAdd,257,257,0,0,EVENT_LOOP,0,0,0_first,258,258,6,47999,EVENT_LOOP,0,0,subscribers,14,StatefulPar
titionedCall/functional_1/max_pool
ing2d/MaxPool_external_StatefulPartitionedCall/functional_1/max_pooling2d/
MaxPool_external,257,257,0,0,UNSAFE_MULTI_EVENT_EMITTER,49417743,0,StatefulPartitionedCall/
functional_1/conv2d_1/BiasAdd_StatefulPartitionedCall/functional_1/conv2d_2/B
i
a
s
A
d
d
_
S
t
a
t
e
f
u
l
P
a
r
t
i
t
i
o
n
e
d
C
a
l
l
/
f
u
n
c
t
i
o
n
a
l
_
1
/
c
o
n
v
2
d
_
1
/
BiasAdd,257,257,0,0,UNSAFE_MULTI_EVENT_EMITTER,0,0,first_StatefulPartitionedCall/functional_1/
conv2d/BiasAdd_first,257,257,3,152213532,UNSAFE_MULTI_EVENT_EMITTER,30926198,0,StatefulPartitio
nedCall/functional_1/conv2d_2/BiasAdd_external_StatefulPartitionedCall/functional_1/conv2d_2/
BiasAdd_external,257,257,0,0,UNSAFE_MULTI_EVENT_EMITTER,0,0,StatefulPartitionedCall/functional_1/
conv2d_3/BiasAdd_external_StatefulPartitionedCall/fu
n
c
t
i
o
n
a
l
_
1
/
c
o
n
v
2
d
_
3
/
BiasAdd_external,257,257,0,0,UNSAFE_MULTI_EVENT_EMITTER,0,0,first_external_first_external,258,258,3,
152252032,UNSAFE_MULTI_EVENT_EMITTER,30911698,0,StatefulPartitionedCall/functional_1/conv2d/BiasAdd-
StatefulPartitionedCal
l/functional_1/max_pooling2d/MaxPool_StatefulPartitionedCall/functional_1/conv2d/
BiasAdd,257,257,2,3026072,UNSAFE_MULTI_EVENT_EMITTER,97849378,0,callback_StatefulPartitionedCall/
functional_1/conv2d_4/BiasAdd,256,256,0,0,UNSAFE_MULTI_EVENT_EMIT
TER,0,0,StatefulPartitionedCall/functional_1/conv2d_4/BiasAdd_external_StatefulPartitionedCall/
f
u
n
c
t
i
o
n
a
l
_
1
/
c
o
n
v
2
d
_
4
/
BiasAdd_external,257,257,0,0,UNSAFE_MULTI_EVENT_EMITTER,0,0,StatefulPartitionedCall/functional_1/
conv2d/BiasAdd_external_Sta
t
e
f
u
l
P
a
r
t
i
t
i
o
n
e
d
C
a
l
l
/
f
u
n
c
t
i
o
n
a
l
_
1
/
c
o
n
v
2
d
/
BiasAdd_external,257,257,2,3041072,UNSAFE_MULTI_EVENT_EMITTER,97838503,0,StatefulPartitionedCall/
functional_1/conv2d_1/BiasAdd_external_StatefulPartitionedCall/functional_1/conv2d_1/
BiasAdd_external,25
7,257,0,0,UNSAFE_MULTI_EVENT_EMITTER,0,0,StatefulPartitionedCall/functional_1/max_pooling2d/MaxPool-
StatefulPartitionedCall/functional_1/conv2d_1/BiasAdd_StatefulPartitionedCall/functional_1/
max_pooling2d/MaxPool,257,257,0,0,UNSAFE_MULTI EVEN
T_EMITTER,49423117,0,StatefulPartitionedCall/functional_1/conv2d_3/BiasAdd_StatefulPartitionedCall/
functional_1/conv2d_4/BiasAdd_StatefulPartitionedCall/functional_1/conv2d_3/
BiasAdd,257,257,0,0,UNSAFE_MULTI_EVENT_EMITTER,0,0,StatefulPartitio
nedCall/functional_1/conv2d_2/BiasAdd_StatefulPartitionedCall/functional_1/conv2d_3/
B
i
a
s
A
d
d
_
S
t
a
t
e
f
u
l
P
a
r
t
i
t
i
o
n
e
d
C
a
l
l
/
f
u
n
c
t
i
o
n
a
l
_
1
/
c
o
n
v
2
d
_
2
/
BiasAdd,257,257,0,0,UNSAFE MULTI EVENT_EMITTER,0,0,

```

Part II: The power of Klepsydra AI

So far, this guide has concentrated on the configuration settings and usability of the libraries. However, it has not yet addressed the performance potential of Klepsydra AI. This chapter will focus on exploring and understanding that aspect.

Please note that the performance improvements discussed here are highly dependent on the specific board used in the tutorial. All the examples provided are based on the Raspberry Pi 3B+ board.

Basic optimisation

The performance improvements presented here are using the basic configuration file, for advance optimisation, i.e., usage of the streaming configuration file, please refer to the SDO user guide included in the documentation pack.

Increase data publishing rate

Let's begin by running the example in production mode:

```
{
  "StringProperties": {
    "container_name": "",
    "log_filename": "",
    "stat_filename": "",
    "processor_intensive_layers": "conv",
    "streaming_configuration": ""
  },
  "IntProperties": {
    "log_level": 2,
    "stat_admin_port": 0,
    "stat_system_port": 0,
    "stat_log_interval_ms": 100,
    "pool_size": 0,
    "number_of_cores": 1,
    "critical_thread_pool_size": 4,
    "non_critical_thread_pool_size": 2,
    "number_of_parallel_threads": 0,
    "first_parallel_core": 0
  },
  "BoolProperties": {
    "use_policy_by_ratio": false,
    "test_mode": false,
    "export_through_tftp": false,
    "export_layer_connections": false,
    "stat_socket_container_enable": false,
    "stat_file_container_enable": false,
    "use_default_streaming_factory": true,
    "export_streaming_configuration": false
  }
}

./kpsr_ai_min_onnx_example -f ../conf/mem_dnn_conf.json -m ../../models/alexnet.onnx -t 100
```

Then you will see some 'no more capacity errors'

```
Applying AI to dummy image 23 of 100
[Tue Jul 9 13:54:42 2024] [13:54:42 960031] [klepsydra] [info] [306]
EventLoopPublisher::internalPublish. no more capacity.first
Image : 23 assigned id : 23
Use this id in callback to match input to result.
[Tue Jul 9 13:54:42 2024] [13:54:42 989620] [klepsydra] [info] [307]
EventLoopPublisher::internalPublish. no more capacity.StatefulPartitionedCall/functional_1/conv2d/
BiasAdd
```

```
[Tue Jul 9 13:54:42 2024] [13:54:42 991018] [klepsydra] [info] [307]
EventLoopPublisher::internalPublish. no more capacity.StatefulPartitionedCall/functional_1/
max_pooling2d/MaxPool
[Tue Jul 9 13:54:43 2024] [13:54:43 038808] [klepsydra] [info] [307]
EventLoopPublisher::internalPublish. no more capacity.StatefulPartitionedCall/functional_1/conv2d/
BiasAdd
[Tue Jul 9 13:54:43 2024] [13:54:43 040117] [klepsydra] [info] [307]
EventLoopPublisher::internalPublish. no more capacity.StatefulPartitionedCall/functional_1/
max_pooling2d/MaxPool
Applying AI to dummy image 24 of 100
[Tue Jul 9 13:54:43 2024] [13:54:43 042205] [klepsydra] [info] [306]
EventLoopPublisher::internalPublish. no more capacity.first
Image : 24 assigned id : 24
Use this id in callback to match input to result.
```

Let's now modify this file as follows:

```
{
  "StringProperties": {
    "container_name": "",
    "log_filename": "",
    "stat_filename": "",
    "processor_intensive_layers": "conv",
    "streaming_configuration": ""
  },
  "IntProperties": {
    "log_level": 2,
    "stat_admin_port": 0,
    "stat_system_port": 0,
    "stat_log_interval_ms": 100,
    "pool_size": 1,
    "number_of_cores": 1,
    "critical_thread_pool_size": 4,
    "non_critical_thread_pool_size": 2,
    "number_of_parallel_threads": 4,
    "first_parallel_core": 0
  },
  "BoolProperties": {
    "use_policy_by_ratio": false,
    "test_mode": false,
    "export_through_tftp": false,
    "export_layer_connections": false,
    "stat_socket_container_enable": false,
    "stat_file_container_enable": false,
    "use_default_streaming_factory": true,
    "export_streaming_configuration": false
  }
}
```

And let's run the example again. Now the log file should not show any more capacity errors.

Advanced optimisation

Further improvements can be made using the streaming configuration. This is what the SDO tool made for, for that, please refer to the SDO documentation.

Appendix A: Configuration

Main configuration file

Users can fine-tune the performance of their CPU applications through various configuration elements. Examples of these configuration files are provided in the tutorial later in this document:

Configuration Variable	Description	Type	Default Value
export_streaming_configuration	Export the default streaming configuration file	Boolean	FALSE
test_mode	Operate in either test mode (asynchronously, without event loops for orchestration) or production mode.	Boolean	TRUE
use_default_streaming_factory	Use the built-in assignation of kernels to event loops during execution	Boolean	TRUE
stat_file_container_enable	Enable statistics gathering on file.	Boolean	FALSE
critical_thread_pool_size	Number of critical threads: Used to configure the event loops. This should be at least equal to the number of event loops.	Integer	2
first_parallel_core	Core id to start the thread pool parallelisation for. E.g., in a 4-core processor, if <code>first_parallel_core = 2</code> , then the pool will be 2 and 3 (0 and 1 are left out of the parallelisation).	Integer	0
log_level	0: trace, 1: debug, 2: info, 3: warn, 4: error	Integer	0
non_critical_thread_pool_size	Number of non-critical threads: Used to configure the schedulers. If statistics are enabled, it should be set to 1; otherwise, it can be 0.	Integer	1
number_of_parallel_threads	Number of parallel threads to be used of parallel execution of layers. Special values are: -1 = no thread pool created, i.e., execution on the same thread as the event loop 0 = all core are used for parallelisation	Integer	0
number_of_cores	Number of cores to be used by the event loops. Usually the have the same value.	Integer	0
number_of_event_loops	This property follows a specific rule: - If <code>number_of_cores` is 0</code> , this property is set to 1 (no cores for orchestration, 1 core for callbacks). - If <code>number_of_cores` is 1</code> , this property is set to 2 (1 core for orchestration, 1 core for callbacks).		0
stat_log_interval_ms	Period in milliseconds for collection of statistics.	Integer	1000
log_filename	Specify the path and filename for writing Klepsydra's internal log files. If left empty, logs will be directed to the standard output.	String	""
container_name	Name of the container to collect statistics to	String	""
stat_filename	Name of the file where to store the statistics (CSV format)	String	""
streaming_configuration	Name of the path and filename to store the default streaming configuration.	String	""

Streaming Configuration

There is a second configuration file for associating kernels with event loops, which is optional and intended for advanced applications only via SDO. The key `layer\_event\_loop\_map` contains a crucial object for mapping middleware providers to CPU cores. Let's break down each of its sub-components:

```
"layer" : { "coreId" : 0, "factoryType" : 1 }
```

The `layer\_event\_loop\_map` is a JavaScript dictionary (or object), where "layer" denotes the key name of the first key-value pair. It signifies the topic name (algorithm step name or Neural network layer name) where publishers connect with subscribers in a middleware provider. You can rename it to a more suitable term if needed. In this example, "layer" is used as neural network layers often correspond to each topic.

The value for each layer in this dictionary is another JSON object, which must include these key names and their respective values:

- `coreId`: An integer that identifies which EventLoop will handle the topic. This corresponds to the `event\_loop\_core\_map` with its name.
- `factoryType`: An integer used to specify the middleware provider for the topic. Options are: 0 for EventEmitter, 1 for EventLoop, 2 for DataMultiplexer, or otherwise an error.

If you intend to create a policy for multiple layers:

```
{
  "pool_size": 0,
  "layer_event_loop_map" : {
    "layer/1" : { "coreId" : 0, "factoryType" : 1 },
    "layer/2" : { "coreId" : 1, "factoryType" : 1 },
    "layer/Emitter" : { "coreId" : 0, "factoryType" : 0 },
  },
  "number_of_cores": 2,
  "non_critical_thread_pool_size": 32,
  "number_of_parallel_threads": 0,
  "parallelised_layers": [],
  "event_loop_core_map": {
    "0": [0],
    "1": [1]
  },
  "number_of_event_loops": 2
}
```

Please note the following considerations:

1. Event Loop Mapping: Ensure `event\_loop\_core\_map` includes an entry to map a second EventLoop (named "1") to `coreId` 1. This setup aligns with `layer\_event\_loop\_map`, where `layer2` will correspond to `coreId` 1.
2. Emitter Configuration: The `coreId` for `layerEmitter` is insignificant, as `factoryType` 0 designates it to run as an EventEmitter.
3. Critical Thread Pool Size: `number\_of\_event\_loops` determines the size of the critical thread pool. Ensure it is not smaller than the number of keys in `event\_loop\_core\_map`.
4. Impact of `number\_of\_cores`: In the current implementation, `number\_of\_cores` does not affect functionality.

Appendix B: Third party installation

Installation of Protobufs

If Protobufs is not previously available on the system, follow these steps to install it locally:

Go to some local folder in the target processor (e.g., \$HOME/thirdparties):

```
sudo apt install autoconf libtool
git clone https://github.com/protocolbuffers/protobuf.git
cd protobuf
git checkout v3.19.0
git submodule update --init --recursive
./autogen.sh && ./configure
make
sudo make install
```

Optional:

```
cd ../
rm -rf protobuf
```

About Klepsydra

Overview

Klepsydra is a Swiss SME founded in 2018 and has currently 12 employees. Under the leadership of CEO Pablo Ghiglino, a team of 9 highly skilled software developers -including 3 PhDs and 1 University professor- is focused on developing high-performance edge data processing software for autonomous embedded systems to serve applications in the space, aerospace, smart mobility and IoT industries. Since its foundation, Klepsydra has received several awards, over 2M CHF in investment and is already in the revenue phase.

Contact

Klepsydra Technologies AG

Brugglenstrasse 2A, 8604, Switzerland.

www.klepsydra.com

sales@klepsydra.com